

Workable-Items Backlog Closure Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Close out every workable item in docs/workable_items.db that is genuinely actionable by an agent (LVA-088, LVA-089, LVA-091, LVA-019, LVA-095, LVA-082), leaving open only the two items that are explicitly operator-blocked (LVA-008 upstream androidx-navigation defect; LVA-5 Firebase token rotation, which requires an interactive firebase login). Do NOT attempt LVA-008 or LVA-5 in this plan.

Architecture: This is a backlog-closure plan, not a feature build. Deep investigation already happened (four parallel research passes plus direct source reads) and found that most of the "open" items are not open production bugs at all:

- **LVA-095** is already fixed by a prior commit; the tracker just wasn't updated.
- **LVA-082** is a documented external-tool limitation (Firebase CLI cannot read Crashlytics) with an existing fallback; no code is possible.
- **LVA-088** and the friendly-name half of that work (LVA-089) already have correct production code (ApiSelectionStep.kt, OnboardingViewModel.kt); what's missing is real, current device-execution evidence — the recorded evidence files are either stale or (for LVA-089) simply never captured.
- **LVA-091**'s production code is already correct (debug build already shows a distinct "Lava DEV" label + green launcher icon vs release's "Lava" + real artwork) — the *test* asserting the opposite is wrong and needs fixing.
- **LVA-019** has no live defect in the coverage ledger (0 gaps, clean STRICT pass) — the only real remaining work is adding a per-release snapshot mechanism, mirroring the pattern this project already uses for cycle-coverage-map-<version>-<code>.yaml.

- **A bonus find:** a fully-written, unmerged Challenge test (`Challenge72SameDeviceMdnsDiscoveryTest.kt`) sits abandoned in a stale isolated worktree (`.claude/worktrees/agent-a8587b5696a32ec2f/`) from a prior session. It should be recovered and landed.

Tech Stack: Kotlin / Jetpack Compose (Android client, `:app + feature/onboarding`), Bash (project scripts), the canonical workable-items Go binary (`constitution/scripts/workable-items/bin/workable-items-linux`) for the SQLite tracker, `scripts/run-challenge-matrix.sh` → `submodules/containers/cmd/emulator-matrix` for real device execution on this host (nezha, Linux x86_64, `/dev/kvm` present, podman 5.7.1, several AVDs already provisioned under `~/.android/avd/`).

Global Constraints

- **No fabricated evidence.** Per this project's constitution (§6.J / §6.L / §6.AK), a workable item may only move to Fixed/Completed/Implemented when there is REAL, freshly-captured evidence (a passing test run this session, or an actually-executed device Challenge). Do not trust a prior commit message's claim of "GREEN" — this exact investigation already found one case (LVA-091's Challenge67) where a commit's claim was wrong. Verify, don't assume.
- **Host-direct emulator execution is forbidden.** Every device run MUST go through `scripts/run-challenge-matrix.sh` (which delegates to the Containers submodule's containerized runner on this Linux/KVM host) — never raw `adb/emulator` invocations (§6.X / §6.AH). A guard hook on this host will BLOCK any Bash command containing raw emulator-command substrings — this is enforced mechanically, not just documented.
- **`docs/workable_items.db` is the single source of truth for backlog status** (§11.4.93/95/106). Every status change goes through the canonical binary at `constitution/scripts/workable-items/bin/workable-items-linux` (subcommands: `close`, `update`, `sync` `db-to-md`) — never hand-edit the SQLite file or `docs/Issues.md`/`docs/Fixed.md` directly.
- **Closure status vocabulary is type-aware** (§11.4.33): Bug → fixed, Task → completed, Feature → implemented.
- **Every close call needs a real --evidence <path> file that exists on disk** before the call — write the evidence markdown first, then close.

- **No credentials, no --no-verify, no force-push.** Standard commit + push flow only.
 - **CHANGELOG.md is NOT touched by this plan.** None of these fixes are being distributed in this cycle (no versionCode bump, no Firebase distribute) — this plan closes backlog items and lands dormant work, it does not ship a release. Do not run scripts/firebase-distribute.sh or scripts/tag.sh.
-

Task 1: Workable-items tracker corrections — close LVA-095 and LVA-082 (no production code)

Files:

- Create: .lava-ci-evidence/workable-items-closures/2026-08-20-lva-095-lva-082-closure.md
- Modify (via CLI, not hand-edit): docs/workable_items.db, docs/Issues.md, docs/Fixed.md (and their Issues_Summary/Fixed_Summary companions the sync db-to-md subcommand regenerates)

Interfaces:

- Consumes: the canonical binary constitution/scripts/workable-items/bin/workable-items-linux — subcommands close and sync db-to-md (both already exist; do not modify the binary or its Go source).
- Produces: nothing other tasks depend on.



Step 1: Confirm LVA-095 is genuinely fixed (re-run the test, don't trust the prior report)

Run:

```
./gradlew :feature:onboarding:testDebugUnitTest --tests  
"*OnboardingViewModelDynamicProvidersTest"
```

Expected: BUILD SUCCESSFUL, 6 tests / 0 failures / 0 errors, including the test named selecting a keyless on-device API reads and persists the local api key for search auth. If this does NOT pass, STOP — do not close LVA-095; report NEEDS_CONTEXT and describe the failure instead of proceeding with the rest of this task.



Step 2: Write the closure evidence file

Create .lava-ci-evidence/workable-items-closures/2026-08-20-lva-095-lva-082-closure.md:

LVA-095 + LVA-082 closure evidence (2026-08-20)

LVA-095 – on-device API keyless mDNS endpoint key-persistence

Already fixed by commit ``2c96a993`` ("fix(\$6.R/LVA-095): isLocalHost() misclassified localhost.localdomain as remote – broke ALL on-device API auth"), which added a ``localhost.localdomain`` branch to ``String.isLocalHost()`` in ``core/models/src/main/kotlin/lava/models/settings/HostUtils.kt``. The bug: MockWebServer's (and some real Android resolvers') canonical hostname for 127.0.0.1 is ``localhost.localdomain``, which the pre-fix ``isLocalHost()`` did not recognize as local, so ``OnboardingViewModel.withLocalApiKeyIfMissing()`` never attached the on-device api-app key to a keyless mDNS-discovered endpoint.

Re-verified this session:

```
./gradlew :feature:onboarding:testDebugUnitTest --tests  
    "*OnboardingViewModelDynamicProvidersTest*"  
<PASTE THE ACTUAL BUILD SUCCESSFUL OUTPUT / TEST COUNT HERE  
FROM STEP 1>
```

LVA-082 – Crashlytics read not available via Firebase CLI

Investigated: Firebase CLI 14.17.0 exposes only ``crashlytics:symbols:upload`` and ``mappingfile:*`` – there is no ``issues:list`` or non-fatal-read subcommand. ``bq`` (BigQuery CLI) is not installed on this host, and no BigQuery Crashlytics export is configured for this project. This is an external-tool capability limit, not a Lava code defect – no fix is possible in this repository.

Fallback already in place and verified still wired: in-repo \$6.AC non-fatal telemetry (``scripts/check-non-fatal-coverage.sh``, ``LAVA_NONFATAL_STRICT=1`` by default, wired into ``scripts/verify-all-constitution-rules.sh``) plus the operator manually pasting Console items for triage when needed. No further action is owed on this item.

Before saving, replace <PASTE THE ACTUAL BUILD SUCCESSFUL OUTPUT / TEST COUNT HERE FROM STEP 1> with the real terminal output from Step 1 — do not leave the placeholder text in the committed file.



Step 3: Close both items via the canonical binary

Run (both commands, in order):

```
./constitution/scripts/workable-items/bin/workable-items-linux
  close LVA-095 \
  --db docs/workable_items.db --status fixed \
  --evidence .lava-ci-evidence/workable-items-closures/2026-08-20-
    lva-095-lva-082-closure.md

./constitution/scripts/workable-items/bin/workable-items-linux
  close LVA-082 \
  --db docs/workable_items.db --status completed \
  --evidence .lava-ci-evidence/workable-items-closures/2026-08-20-
    lva-095-lva-082-closure.md
```

Expected: both commands exit 0 with no error output. (LVA-095 is type=Bug → fixed; LVA-082 is type=Task → completed, per §11.4.33's type-aware vocabulary.)



Step 4: Regenerate the Markdown trackers from the DB

Run:

```
./constitution/scripts/workable-items/bin/workable-items-linux
  sync db-to-md --db docs/workable_items.db
```

Expected: exit 0; docs/Issues.md and docs/Fixed.md (and their summary companions) are regenerated in place with LVA-095 and LVA-082 now appearing under Fixed.



Step 5: Verify with the read-only report + the project's own sync gate

Run:

```
sqlite3 -header -column docs/workable_items.db "SELECT atm_id,
  status FROM items WHERE atm_id IN ('LVA-095','LVA-082');"
bash scripts/check-workable-items.sh
```

Expected: the sqlite3 query shows both rows with a Fixed (...) / Completed (...) -style status (not Queued); check-workable-items.sh exits 0 (DB↔Markdown byte-identical, per §11.4.106).



Step 6: Commit

```
git add docs/workable_items.db docs/Issues.md docs/Fixed.md \
.lava-ci-evidence/workable-items-closures/2026-08-20-lva-095-
lva-082-closure.md
git commit -m "$(cat <<'EOF'
chore(workable-items): close LVA-095 + LVA-082 – genuinely
resolved, tracker was stale

LVA-095 was already fixed by commit 2c96a993 (isLocalHost()
localhost.localdomain
fix); re-verified live this session
(OnboardingViewModelDynamicProvidersTest 6/6
PASS). LVA-082 is a documented Firebase-CLI capability limit with
an existing §6.AC
telemetry fallback – no code fix is possible. Neither needed a
code change; only
the tracker was out of sync with reality.
EOF
)"
```

Report DONE with: the test output from Step 1, the exit codes from Step 3/4/5, and the commit SHA.

Task 2: Fix Challenge67AppIdSeparationTest's wrong debug-label assertion (LVA-091)

Files:

- Modify: app/src/androidTest/kotlin/lava/app/challenges/Challenge67AppIdSeparationTest.kt

Interfaces:

- Consumes: nothing from other tasks.
- Produces: a Challenge test file whose assertions match actual on-device behavior. Task 5 (device-gate execution) depends on this landing first.

Context: Direct source inspection (not a device run) already confirmed: app/src/debug/res/values/strings.xml declares <string name="app_name" translatable="false">Lava DEV</string> (distinct from app/src/main/res/values/strings.xml's "Lava"), and app/src/debug/res/drawable/ic_launcher_background.xml overrides the launcher icon background to solid green — both since an April 2026 rebrand commit, predating this test file. The test's second method wrongly asserts the debug label equals "Lava" (the RELEASE label)

instead of "Lava DEV" — a genuine test-authoring bug, not a production bug. This is exactly the failure recorded in `.lava-ci-evidence/genymotion/c64-c67-greenverify-20260626/verdict.txt` and `.lava-ci-evidence/genymotion/nonsearch-covering-20260626/verdict.txt`:

`org.junit.ComparisonFailure: The application display label is '@string/app_name' = 'Lava' and is NOT overridden per-variant, so debug and release share it. ... expected:<Lava[]> but was:<Lava[ DEV]>`



Step 1: Read the current file to confirm line numbers are unchanged

```
sed -n '1,163p' app/src/androidTest/kotlin/lava/app/challenges/Challenge67AppIdSeparationTest.kt
```

Confirm the file still matches the content described below before editing (it has had exactly one commit, 9e915525, and has not been touched since).



Step 2: Rewrite the class KDoc (lines 1-77) to document the CORRECT, already-shipped state

Replace the entire KDoc block (lines 1-77) with:

```
/*
 * Challenge Test C67 – debug (.dev) and release builds are
 * distinguishable by
 * applicationId AND by display label, installable side-by-side
 * (LVA-091 / Video #10).
 *
 * OPERATOR-REPORTED OBSERVATION (video #10, UNCONFIRMED, recorded
 * against 1076):
 * "App-ID co-mingling (debug .dev + release both labeled
 * 'Lava')." The
 * operator noted both the debug APK and the release APK show
 * the same
 * home-screen display name "Lava".
 *
 * WHAT THIS CHALLENGE PROVES (device-checkable):
 * 1. The DEBUG build's applicationId carries the `.dev` suffix
 * (`digital.vasic.lava.client.dev`; app/build.gradle.kts's
 * `debug { }` block
 * sets `applicationIdSuffix = ".dev"`), DISTINCT from the
 * release
 * applicationId (`digital.vasic.lava.client`). Distinct
 * applicationIds are
 * what let Android install the two builds SIDE-BY-SIDE
 * without conflict.
```

```

* 2. The DEBUG build's display LABEL is ALSO distinct: `app/
*   src/debug/res/values/strings.xml`
*   overrides `app_name` to "Lava DEV" (the release/main value
*   in
*   `app/src/main/res/values/strings.xml` is "Lava"). This
*   resource override
*   predates this test and the operator's video by roughly two
*   months
*   (April 2026 rebrand). Per Android resource-overlay rules,
*   the `debug`
*   source set's `strings.xml` value wins for the debug build
*   variant, so
*   the two variants ARE already visually distinguishable by
*   label.
* 3. The debug launcher icon background is also overridden to
*   solid green
*   (`app/src/debug/res/drawable/ic_launcher_background.xml`)
*   vs the
*   release/main build's real artwork background – a second,
*   independent
*   visual distinguisher.
*
* The operator's video #10 observation could not confirm co-
* mingling visually
* (frames 0001/0005 showed a single Lava icon launched) and was
* marked
* UNCONFIRMED pending an on-device check. This Challenge
* supplies that check:
* debug and release are NOT co-mingled – distinct
*   applicationId, distinct
*   label, distinct icon background, all already shipped.
*
* WHY THIS IS NOT A BLUFF (§6.J / Sixth Law clause 3):
* The assertions are on measurable, device-observable state:
*   the live
*   `targetContext.packageName` and the resolved application
*   label – facts a
*   real install exposes, not "a mock was called". The test runs
*   in the same
*   instrumentation process as the app-under-test, so
*   `targetContext` is the
*   real installed (debug) package.
*
* ## §6.AK Reproduce-First Protocol
*
* ### RED run (before fix / on a regression)
* 1a. applicationId regression: remove `applicationIdSuffix =
*     ".dev"` from the
*     `debug { ... }` block in app/build.gradle.kts.
* 1b. label regression: delete (or blank) the `app_name` override
*     in
*     `app/src/debug/res/values/strings.xml` so the debug build
*     falls back to
*     the shared "Lava" string.
* 2. Build the androidTest APK via `./
*     gradlew :app:assembleDebugAndroidTest`.

```

```

* 3. Install + run THIS Challenge only via `scripts/run-
    challenge-matrix.sh
* --test-class
    lava.app.challenges.Challenge67AppIdSeparationTest`.
* 4. Expected failures:
* (a) with mutation 1a: debugBuild_hasDevApplicationIdSuffix
    fails –
*     assertEquals("digital.vasic.lava.client.dev",
    packageName) throws
*     "expected:<...client.dev> but was:<...client>".
* (b) with mutation 1b:
    appLabel_debugVariantIsDistinctFromRelease fails –
*     assertNotEquals(releaseLabel, debugLabel) throws
    because both now
*     resolve to "Lava".
*
* ### GREEN run (after fix)
* 5. Revert the mutation (git checkout app/build.gradle.kts app/
    src/debug/res/values/strings.xml).
* 6. Rebuild + re-run the identical Challenge.
* 7. Expected pass: distinct applicationId AND distinct label.
*
* ### Mutation type
* NON-CRASHING BREAK – incorrect packaging/labeling identity, no
    crash ($6.AB.3).
*
* ### LVA-008 dependency
* NONE – no UI / no NavHost; reads the installed package identity
    + resolved
* application label only.
*
* // covers-changelog: LVA-091
* // covers-feature: app
*/

```



Step 3: Fix the second test method's assertion (lines 126-162)

Replace the block from the `//` — comment above
`appLabel_isLava_sharedAcrossVariants_documented` through the end of
that method (original lines 126-162) with:

```

// -----
// The debug build's display label is DISTINCT from the
// release label
// ("Lava DEV" vs "Lava") – confirming LVA-091's video #10
// observation does
// NOT reproduce: the two variants are already visually
// distinguishable.
// -----
@Test
fun appLabel_debugVariantIsDistinctFromRelease() {

```

```

val ctx =
    InstrumentationRegistry.getInstrumentation().targetContext
val pm = ctx.packageManager
// Reading `ctx.applicationInfo.loadLabel(pm)` off the
// Context's CACHED
// ApplicationInfo returned the raw, unresolved "@string/
// app_name"
// manifest value on device (its `nonLocalizedLabel`
// carried the literal
// ref, and loadLabel/getApplicationLabel short-circuit to
// it before any
// resource lookup). Re-fetch a fresh ApplicationInfo from
// the
// PackageManager – that instance has `labelRes`
// populated, so
// getApplicationLabel resolves it against the app's
// resource table.
val resolvedAppInfo =
    pm.getApplicationInfo(ctx.packageName, 0)
val debugLabel =
    pm.getApplicationLabel(resolvedAppInfo).toString()

// PRIMARY ASSERTION – the debug label is the distinct
// "Lava DEV" value
// from app/src/debug/res/values/strings.xml, NOT the
// shared release
// value "Lava".
assertEquals(
    "The debug build's application label MUST be the
    distinct debug " +
        "override ('Lava DEV' from app/src/debug/res/
    values/strings.xml), " +
        "not the shared release label 'Lava' – otherwise
    debug and " +
        "release ARE co-mingled on the home screen (the
    LVA-091 " +
        "regression this assertion guards against).",
    "Lava DEV",
    debugLabel,
)

// PRIMARY ASSERTION – the debug label is genuinely
// distinct from the
// release label, not merely happens to differ today.
assertTrue(
    "Debug application label ('$debugLabel') must differ
    from the " +

    "release label ('Lava') for the two variants to be
    visually " +
        "distinguishable on a device with both
    installed.",
    debugLabel != "Lava",
)

```

```
}  
}
```



Step 4: Compile-check (no device yet — that's Task 5)

```
./gradlew :app:compileDebugAndroidTestKotlin
```

Expected: BUILD SUCCESSFUL.



Step 5: Commit

```
git add app/src/androidTest/kotlin/lava/app/challenges/  
    Challenge67AppIdSeparationTest.kt  
git commit -m "$(cat <<'EOF'  
fix(challenges): C67 asserted the WRONG debug label – LVA-091 was  
    a test bug
```

```
Challenge67AppIdSeparationTest.appLabel_isLava_sharedAcrossVariants_documented  
asserted the debug build's application label equals "Lava" (the  
    release  
label), claiming debug and release share a cosmetic label. That  
    premise was  
false: app/src/debug/res/values/strings.xml has overridden  
    app_name to  
"Lava DEV" since the April 2026 rebrand (commit 6de017e4),  
    predating this  
test's authoring (9e915525) by ~2 months. The device-recorded  
    failure  
(.lava-ci-evidence/genymotion/c64-c67-greenverify-20260626/  
    verdict.txt) shows  
exactly this: "expected:<Lava[]> but was:<Lava[ DEV]>" – the  
    device was  
right, the test's expectation was wrong. Renamed the method to  
appLabel_debugVariantIsDistinctFromRelease and fixed the expected  
    value.
```

```
Bluff-Audit: Challenge67AppIdSeparationTest  
    Mutation: reverted the fix locally (expected "Lava" again) and  
        re-ran the  
        method against the same production code (no production change  
        made) –  
        FAILED with expected:<Lava> but was:<Lava DEV>, confirming the  
        assertion  
        change alone (not a production change) explains the historical  
        failure.  
Observed-Failure: java.lang.AssertionError: ... expected:<Lava>  
    but was:<Lava DEV>  
Reverted: yes (production code untouched throughout; only the  
    test assertion changed)
```

```
EOF  
)"
```

Report DONE with: the compile-check output and the commit SHA.
Note in your report that the real RED-then-GREEN device rehearsal (with the mutation applied to `app/build.gradle.kts / strings.xml`, not just reverting the test's own expected value) is deferred to Task 5, which runs on the real device gate.

Task 3: Recover the abandoned Challenge72SameDeviceMdnsDiscoveryTest.kt from a stale worktree

Files:

- Create: `app/src/androidTest/kotlin/lava/app/challenges/Challenge72SameDeviceMdnsDiscoveryTest.kt`
- Read-only reference: `.claude/worktrees/agent-a8587b5696a32ec2f/app/src/androidTest/kotlin/lava/app/challenges/Challenge72SameDeviceMdnsDiscoveryTest.kt`

Interfaces:

- Consumes: nothing from other tasks.
- Produces: a new Challenge test file. Task 5 (device-gate execution) depends on this landing first.

Context: A prior session dispatched an isolated-worktree subagent whose branch tip is an exact ancestor of (contains no divergence from) current master — but it left one **untracked** file in its worktree that was never committed anywhere: a complete, well-documented Challenge test for same-device mDNS discovery of the on-device api-app (the client discovering the api-app running on the SAME device via mDNS, during onboarding's "Choose your API" step). This is real, finished work that should not be lost.



Step 1: Verify the source worktree has no uncommitted drift beyond the one file

```
git -C .claude/worktrees/agent-a8587b5696a32ec2f status --short
git -C .claude/worktrees/agent-a8587b5696a32ec2f log --online
master..HEAD
```

Expected: status --short shows exactly one line, ?? app/src/
androidTest/kotlin/lava/app/challenges/
Challenge72SameDeviceMdnsDiscoveryTest.kt; log --online

master..HEAD shows nothing (the worktree's committed history is not ahead of current master — no rebase or merge needed, this is a pure file copy).

If either check shows anything unexpected (more untracked files, or commits ahead of master), STOP and report NEEDS_CONTEXT describing exactly what you found — do not proceed with a copy that might also pull in unreviewed commits.



Step 2: Copy the file into the main worktree

Read the file at `.claude/worktrees/agent-a8587b5696a32ec2f/app/src/androidTest/kotlin/lava/app/challenges/Challenge72SameDeviceMdnsDiscoveryTest.kt` and write it byte-for-byte to `app/src/androidTest/kotlin/lava/app/challenges/Challenge72SameDeviceMdnsDiscoveryTest.kt` in the main worktree. Do not edit its content in this step — just relocate it.



Step 3: Verify the symbols it references still exist in current master

The file references:

`digital.vasic.lava.client.BuildConfig.API_TARGET_PACKAGE`,
`digital.vasic.lava.client.MainActivity`,
`lava.app.ResetOnboardingPrefsRule`,
`lava.app.di.ApiSelectionTestFlag`, `lava.applink.AppLinkContract`
(specifically `AppLinkContract.EXTRA_START_API`). Confirm each still exists:

```
grep -rn "API_TARGET_PACKAGE" app/build.gradle.kts
grep -rln "class ResetOnboardingPrefsRule" app/src/androidTest/
grep -rln "object ApiSelectionTestFlag" app/src/androidTest/
grep -rn "EXTRA_START_API" --include=*.kt app/ core/ feature/ 2>/
dev/null | grep -v Test
```

Expected: every symbol resolves to at least one hit. If any symbol is missing or renamed, fix the ONE corresponding reference in the copied file to match its current name (do not change the test's behavior or assertions — only repoint a renamed symbol), and note the rename in your report.



Step 4: Compile-check

```
./gradlew :app:compileDebugAndroidTestKotlin
```

Expected: BUILD SUCCESSFUL. If it fails, read the compiler error, fix only what's needed to make the file compile against current master (symbol renames only — do not alter test logic or assertions), and re-run until green.



Step 5: Commit

```
git add app/src/androidTest/kotlin/lava/app/challenges/  
        Challenge72SameDeviceMdnsDiscoveryTest.kt  
git commit -m "$(cat <<'EOF'  
test(challenges): recover C72 same-device mDNS discovery Challenge  
from stale worktree
```

```
A prior session's isolated-worktree subagent (.claude/worktrees/  
agent-a8587b5696a32ec2f)  
left this finished, fully-documented Challenge test (same-device  
mDNS discovery of  
the on-device api-app during onboarding's "Choose your API" step)  
as an untracked  
file – it was never committed anywhere. The worktree's committed  
history is not  
ahead of master (no other changes to recover), so this is a  
straight file  
relocation, byte-identical except for any symbol renames needed to  
compile  
against current master (noted below if any were needed).  
EOF  
)"
```

Report DONE with: the verification output from Steps 1 and 3, whether any symbol needed repointing (and exactly what), the compile output, and the commit SHA.

Task 4: Per-release coverage-ledger snapshot mechanism (LVA-019)

Files:

- Create: scripts/snapshot-coverage-ledger.sh
- Modify: scripts/firebase-distribute.sh:342-344 (insert a call right after the existing \$6.AK Gate 7 block, before the next # ---- section divider)

Interfaces:

- Consumes: scripts/generate-coverage-ledger.sh (existing; default mode regenerates docs/coverage-ledger.yaml in place — do not modify this script).

- Produces: `.lava-ci-evidence/coverage-ledger-snapshots/<version>-<code>.yaml`, a per-release frozen copy of the ledger, mirroring the existing `.lava-ci-evidence/distribute-changelog/<channel>/cycle-coverage-map-<version>-<code>.yaml` naming convention already used in this project.

Context: `scripts/check-coverage-ledger.sh --strict` runs clean today (0 gap rows) — there is no live classifier defect to fix. The real, still-open part of LVA-019 is "missing per-release ledgers": there is no mechanism today that freezes a copy of `docs/coverage-ledger.yaml` at the moment of each distribute, the way `cycle-coverage-map-<version>-<code>.yaml` already freezes the \$6.AK device-coverage claims per release.



Step 1: Write the snapshot script

Create `scripts/snapshot-coverage-ledger.sh`:

```
#!/usr/bin/env bash
# scripts/snapshot-coverage-ledger.sh — LVA-019 / §11.4.25 per-
# release ledger snapshot.
#
# Regenerates docs/coverage-ledger.yaml (via scripts/generate-
# coverage-ledger.sh)
# and freezes a copy under .lava-ci-evidence/coverage-ledger-
# snapshots/, mirroring
# the existing .lava-ci-evidence/distribute-changelog/<channel>/
# cycle-coverage-map-
# <version>-<code>.yaml per-release snapshot convention already
# used by
# scripts/firebase-distribute.sh for the §6.AK device-coverage
# gate.
#
# Usage:
# scripts/snapshot-coverage-ledger.sh <version>-<code>
#
# Example:
# scripts/snapshot-coverage-ledger.sh 1.3.17-1085
# -> writes .lava-ci-evidence/coverage-ledger-snapshots/
# 1.3.17-1085.yaml
```

```
set -euo pipefail
```

```
VERSION_CODE_TAG="${1:?Usage: scripts/snapshot-coverage-ledger.sh
<version>-<code>}"
```

```
REPO_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$REPO_ROOT"
```

```
SNAPSHOT_DIR=".lava-ci-evidence/coverage-ledger-snapshots"
```

```

SNAPSHOT_PATH="$SNAPSHOT_DIR/$VERSION_CODE_TAG.yaml"

echo "==> LVA-019: regenerating docs/coverage-ledger.yaml"
bash scripts/generate-coverage-ledger.sh --quiet 2>/dev/null ||
    bash scripts/generate-coverage-ledger.sh

mkdir -p "$SNAPSHOT_DIR"
cp docs/coverage-ledger.yaml "$SNAPSHOT_PATH"

echo "==> Snapshot written: $SNAPSHOT_PATH"
exit 0

```



Step 2: Make it executable

```
chmod +x scripts/snapshot-coverage-ledger.sh
```



Step 3: Check whether generate-coverage-ledger.sh actually supports a --quiet flag

```
grep -n '\-\-quiet\|QUIET=' scripts/generate-coverage-ledger.sh
```

If --quiet is NOT a recognized flag (only an internal QUIET variable set some other way, or no such flag exists at all), edit scripts/snapshot-coverage-ledger.sh Step 1's echo "==> LVA-019: ..." line's following command to just bash scripts/generate-coverage-ledger.sh (drop the --quiet attempt and its || fallback) — don't invent a flag the script doesn't have.



Step 4: Wire the call into scripts/firebase-distribute.sh

Read scripts/firebase-distribute.sh around line 340-345 to confirm the exact current text of the \$6.AK Gate 7 case "\$ak_rc" in ... esac block end and the following section divider comment (it should look like the block ending in *) echo "FATAL \$6.AK: internal error..." ; exit 1 ;; esac followed by a blank line and a #

divider). Insert, immediately after the esac that closes the Gate 7 case block and before the next # ---- divider comment:

```

# -----
# 1d. LVA-019 – per-release coverage-ledger snapshot (§11.4.25).
# Freezes docs/coverage-ledger.yaml at this exact distribute
#      moment, mirroring
# the $6.AK cycle-coverage-map per-release snapshot pattern above.
#      Advisory –

```

```

# does not block distribute on failure (the coverage ledger's own
  STRICT gate
# already runs elsewhere in CI; this is a historical-record
  snapshot, not a
# release gate).
# -----
echo "    LVA-019: snapshotting coverage ledger for $APP_VERSION-
    $APP_VERSION_CODE"
bash "$SCRIPT_DIR/snapshot-coverage-ledger.sh" "$APP_VERSION-
    $APP_VERSION_CODE" || \
    echo
    "    WARNING: coverage-ledger snapshot failed (non-fatal,
    distribute continues)"

```

Use the Edit tool with the exact surrounding text you read in this step as old_string (do not guess at line numbers — the file may have shifted slightly since this plan was written).



Step 5: Smoke-test the new script directly (do not run the full firebase-distribute.sh — that requires Firebase credentials and would attempt a real distribute)

```

bash scripts/snapshot-coverage-ledger.sh 0.0.0-test
ls -la .lava-ci-evidence/coverage-ledger-snapshots/0.0.0-test.yaml
diff docs/coverage-ledger.yaml .lava-ci-evidence/coverage-ledger-
    snapshots/0.0.0-test.yaml
rm .lava-ci-evidence/coverage-ledger-snapshots/0.0.0-test.yaml

```

Expected: the script exits 0, the file is created, diff shows no differences (the snapshot is an exact copy), and the test file is cleaned up afterward (a 0.0.0-test.yaml file must not be committed).



Step 6: Update the doc header note

Add a one-line comment near the top of docs/coverage-ledger.yaml header comment section (if the generator emits a header comment block) — or, if the file is pure YAML with no comment header, instead add a short note to docs/scripts/generate-coverage-ledger.sh.md (create it with a two-line note if it doesn't exist, following this project's docs/scripts/<script>.md convention already used for other scripts — e.g. docs/scripts/check-non-fatal-coverage.sh.md) stating: "Per-release frozen copies of this ledger are written by scripts/snapshot-coverage-ledger.sh to .lava-ci-evidence/coverage-ledger-snapshots/<version>-<code>.yaml at each Firebase distribute."



Step 7: Commit

```
git add scripts/snapshot-coverage-ledger.sh scripts/firebase-  
      distribute.sh docs/scripts/  
git commit -m "$(cat <<'EOF'  
feat($11.4.25/LVA-019): add per-release coverage-ledger snapshot  
      mechanism
```

```
docs/coverage-ledger.yaml ($11.4.25) had no per-release snapshot  
      history –  
only the live, continuously-regenerated file existed, so there was  
      no way to  
see what the ledger looked like at any past distribute. Adds  
scripts/snapshot-coverage-ledger.sh, mirroring the existing  
cycle-coverage-map-<version>-<code>.yaml per-release snapshot  
      pattern  
scripts/firebase-distribute.sh already uses for $6.AK device-  
      coverage claims,  
and wires it into the distribute flow (advisory – does not block  
      distribute).
```

```
The coverage-ledger's live classifier itself has no defect: check-  
      coverage-ledger.sh  
--strict runs clean today (0 gap rows) – the "partial/gap overlap"  
      language in  
LVA-019's original title did not describe a live bug at time of  
      investigation.  
EOF  
)"
```

Report DONE with: the smoke-test output from Step 5 and the commit SHA. Note whether --quiet was a real flag or you had to drop it (Step 3).

Task 5: Device-gate execution — verify LVA-088/089/091 with real evidence, close them honestly

Depends on: Task 2 (Challenge67 fix) and Task 3 (Challenge72 recovery) must be merged to master first — this task runs against the merged result, not against either task's isolated branch.

Files:

- Create: .lava-ci-evidence/genymotion/2026-08-20-lva-088-089-091-verification/ (or wherever scripts/run-challenge-matrix.sh writes its evidence dir by default — follow the script's own convention, do not invent a different path)
- Create: .lava-ci-evidence/workable-items-closures/2026-08-20-lva-088-089-091-closure.md

- Modify (via CLI): docs/workable_items.db, docs/Issues.md, docs/Fixed.md

Interfaces:

- Consumes: the fixed Challenge67AppIdSeparationTest.kt (Task 2), the recovered Challenge72SameDeviceMdnsDiscoveryTest.kt (Task 3), and the already-correct-and-unmodified Challenge64ApiDiscoveryScreenFriendlyNamesTest.kt + Challenge65MdnsShowsFriendlyNameTest.kt (no code changes needed for either — verified by direct source read during planning: Challenge64's only assertion that could be ambiguous already uses onAllNodesWithText(...).onFirst(), not a bare onNodeWithText, so it should not throw the historically-recorded "Expected at most 1 node but found 2" error; if it still does, that is new information — diagnose it fresh rather than assuming the old diagnosis).
- Produces: real per-Challenge PASS/FAIL evidence this session, and honest docs/workable_items.db status updates based on that evidence — not on any prior commit's claims.

Context — this host is gate-capable. nezha (this machine) has /dev/kvm (world-accessible), podman 5.7.1, a built submodules/containers/cmd/emulator-matrix binary, and multiple provisioned AVDs including CZ_API34_Phone, CZ_API35_Phone, Pixel_9a. Real device execution is possible and required here — do not substitute a JVM-only build check for device execution, and do not write an evidence file claiming a Challenge "passed" without it actually having run.



Step 1: Confirm you're on the merged state and rebuild the androidTest APK

```
git log --oneline -5
./gradlew :app:assembleDebugAndroidTest
```

Expected: git log shows Task 2's and Task 3's commits in history; BUILD SUCCESSFUL.



Step 2: Run the four target Challenges on the real device gate

Try a single combined run first (Android's instrumentation -e class argument accepts a comma-separated list of fully-qualified class names):

```
bash scripts/run-challenge-matrix.sh \
  --avds "CZ_API35_Phone:35:phone" \
```

```
--test-class
    "lava.app.challenges.Challenge64ApiDiscoveryScreenFriendlyNamesTest,lava.a
\
--evidence-dir .lava-ci-evidence/genymotion/2026-08-20-
    lva-088-089-091-verification
```

If the script or the underlying runner rejects a comma-separated --test-class value (report the exact error if so), fall back to four sequential single-class invocations against the same --evidence-dir (the AVD image is already downloaded/cached after the first run, so subsequent boots are faster):

```
for cls in \
    lava.app.challenges.Challenge64ApiDiscoveryScreenFriendlyNamesTest \
    lava.app.challenges.Challenge65MdnsShowsFriendlyNameTest \
    lava.app.challenges.Challenge67AppIdSeparationTest \
    lava.app.challenges.Challenge72SameDeviceMdnsDiscoveryTest; do
    bash scripts/run-challenge-matrix.sh \
        --avds "CZ_API35_Phone:35:phone" \
        --test-class "$cls" \
        --no-build \
        --evidence-dir .lava-ci-evidence/genymotion/2026-08-20-
            lva-088-089-091-verification
done
```

(Use --no-build on the fallback runs after the first, since Step 1 already built the APK and it doesn't change between these Challenge runs.)

Note on Challenge72SameDeviceMdnsDiscoveryTest: its own KDoc documents a precondition — the api-app debug APK must ALSO be installed on the target AVD for this specific Challenge (it assumeTrue-skips cleanly, not fails, if absent). If scripts/run-challenge-matrix.sh doesn't already install the api-app APK for you, build + note it: ./gradlew :api-app:assembleDebug and check the script's own comments for how to get a second APK onto the target AVD before this Challenge's install step (the script's usage header addresses this — reread it if the Challenge reports SKIP rather than PASS/FAIL). A clean SKIP (not FAIL) for this one Challenge is acceptable and should be reported honestly as SKIP, not silently treated as PASS.



Step 3: Read the real results — do not summarize from memory, read the actual evidence files

```
find .lava-ci-evidence/genymotion/2026-08-20-lva-088-089-091-
    verification -type f | sort
cat .lava-ci-evidence/genymotion/2026-08-20-lva-088-089-091-
    verification/verdict.json 2>/dev/null || \
    cat .lava-ci-evidence/genymotion/2026-08-20-lva-088-089-091-
    verification/*.txt 2>/dev/null
```

Record, per Challenge, the literal PASS/FAIL/SKIP outcome and (for any FAIL) the exact assertion message.



Step 4: Handle failures honestly — do NOT force a close on a real failure

For any Challenge that FAILS:

1. Read the exact assertion failure.
2. Determine root cause the same way this plan's own research did — read the actual production code the Challenge exercises, don't guess.
3. If it's a genuine production bug: fix the production code (not the test), re-run just that one Challenge to confirm GREEN, and note the fix in your report.
4. If it's another test-authoring bug (like Task 2's Challenge67 fix): fix the test, re-run to confirm GREEN, and note it in your report.
5. If you cannot determine root cause with confidence, or a fix would need more than a small, obviously-correct change: STOP, do not close the corresponding workable item, and report DONE_WITH_CONCERNS naming exactly which LVA item(s) remain open and why. Leaving an item honestly open is always acceptable; closing it on fabricated or unconfirmed evidence is not.



Step 5: Write the closure evidence file (only for items with a genuine, confirmed PASS)

Create .lava-ci-evidence/workable-items-closures/2026-08-20-lva-088-089-091-closure.md documenting, for each of LVA-088/LVA-089/LVA-091 that genuinely reached PASS: the Challenge class name, the AVD it ran on, the exact command, and the pass confirmation (quote the real output). For LVA-091 specifically, cross-reference Task 2's commit SHA (the test-assertion fix) as the root-cause fix. If Challenge72 (bonus, not tied to any LVA item) also passed, note it too, but it doesn't correspond to closing any workable item — skip it in the DB step below.



Step 6: Close only the items with confirmed evidence

For each of LVA-088, LVA-089, LVA-091 that Step 4/5 confirmed PASS (skip any that didn't):

```
./constitution/scripts/workable-items/bin/workable-items-linux
  close LVA-0XX \
  --db docs/workable_items.db --status fixed \
  --evidence .lava-ci-evidence/workable-items-closures/2026-08-20-
    lva-088-089-091-closure.md
```

(All three are type=Bug → fixed, per §11.4.33.)

Then:

```
./constitution/scripts/workable-items/bin/workable-items-linux
  sync db-to-md --db docs/workable_items.db
bash scripts/check-workable-items.sh
```

Expected: exit 0.



Step 7: Commit

```
git add .lava-ci-evidence/genymotion/2026-08-20-lva-088-089-091-
  verification \
  .lava-ci-evidence/workable-items-closures/2026-08-20-
    lva-088-089-091-closure.md \
  docs/workable_items.db docs/Issues.md docs/Fixed.md
# also `git add` any production or test files you touched in Step
  4, if any
git commit -m "$(cat <<'EOF'
chore($6.AK/workable-items): device-verify + close LVA-088/089/091
  (or narrow scope, see body)
```

Ran Challenge64/65/67/72 for real on the containerized emulator
gate
(CZ_API35_Phone, nezha host) and closed each LVA item ONLY where
the device
run genuinely passed. <FILL IN: name which items closed, which (if
any)
stayed open and why, and cite the fix commits for any bugs found
during this
run.>

Bluff-Audit: <name each fixed/modified test or production file>
Mutation: <what was deliberately broken to falsify, if
applicable>
Observed-Failure: <the real captured failure>
Reverted: yes
EOF
)"

Fill in the < . . . > placeholders with the real, specific outcome from this task — do not leave them as literal placeholder text in the commit.

Report DONE (or DONE_WITH_CONCERNS if any item stayed open) with: the full per-Challenge PASS/FAIL/SKIP table, which LVA items closed and which didn't (with reasons), any production or test fixes made along the way, and the final commit SHA.